D: Processor

Descriere

A computer processor decodes an instruction and its operands and executes it by analyzing a stream of bits. A simplified version of a processor implementation can be found in `process_bits.c`. The program reads multiple decimal numbers: the first number is a 32 bit integer and it represents the instruction codification. The rest are 16 bit integers and represent the operands. The layout of an instruction, which is represented by the first number, is as follows:

1. The first 3 bits encode the number of instructions in the program, **N**. **N** is obtained by converting these first 3 bits to decimal and adding `1`. For example, if the first 3 bits of the instruction are 010, this means we have to execute 3 instructions.

2. The following `N*2` bits represent the operations, **Op**, that should be performed. Following are the 4 possible operations and their encoding:
    - `+` - `00`
    - `-` - `01`
    - `*` - `10`
    - `/` - `11`

3. The following 4 bits represent the dimension of the operands, **Dim**. **Dim** is computed the same way as **N**: convert the bits to decimal and add 1. The dimension can only be a power of 2: 1, 2, 4, …, 16.

The subsequent numbers that are represent the operands and should be interpreted based on the previously read instruction. The type of these numbers is `unsigned short`. For example, if the dimension is considered to be 2 and there are 3 operations involved, this means that a single unsigned short number will suffice to represent the operands: just 8 bits are needed to represent the 4 operands. If we have 4 operations and the dimension of an operand is 4, then we will need 2 `unsigned short` numbers (5 operands * 4 (dimension) = 20). 4 operands are stored in the first number, while the 5th is stored in the first 4 bits of the second number. The remaining bits of the second number are set to 0.

The processor does not implement operator precedence, therefore, the operations are performed in the order that they are given. `1 + 2 * 3` will result in 9, not 7.

The supplied program should follow all of these rules. However, a bug seems to have crept in. Solve it!

Download the source file here.

Examples & explanations :

Example 1

```
1410859008 54999

1410859008 -> binary representation: 010 1010 00 0011 0000000000000000
First 3 bits: 010 => N = 2 + 1 = 3
Next 3*2 bits: 10 10 00 => Ops: * * +
Next 4 bits: 0011 => Dim = 3 + 1 = 4
Instruction decoding: 3 * * + 4

54999 -> binary representation:  1101 0110 1101 0111
Since Dim = 4, the operands are 4 bit numbers
Operands: 13 6 13 7
```

```
The expression to be evaluated: 13 * 6 * 13 + 7
Result: 1021
```

Example 2

```
1947074560 54998 64041 42752

1947074560 -> binary representation: 011 10 10 00 00 0111 1000000000000000
First 3 bits: 011 => N = 3 + 1 = 4
Next 4*2 bits: 10 10 00 00 => Ops: * * + +
Next 4 bits: 0111 => Dim = 7 + 1 = 8
Instruction decoding: 4 * * + + 8

54998 -> binary representation: 11010110 11010110
64041 -> binary representation: 11111010 00101001
42752 -> binary representation: 10100111 00000000

Since Dim = 8, the operands are 8 bit numbers
Operands: 214 214 250 41 167
The expression to be evaluated: 214 * 214 * 250 + 41 + 167
Result: 11449208
```

Exemple